

VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System

Infosys Springboard Internship 7.0

Milestone 1

By Pathlavath Vasu Nayak From *Team -1*

Table of Contents

Title	No	Page No
Project Description	1	1
Dataset Used	2	2
Environment Setup	3	2
System Architecture & RBAC	4	4
Core Implementation	5	8
Verification & Results	6	9
Conclusion	7	9

1 Project Description

The VisionInspect AI system is an end-to-end, enterprise-grade machine learning and computer vision platform designed to automate quality control and defect detection in Industry 4.0 smart manufacturing environments. The primary goal of this project is to replace manual visual inspection with an automated, high-throughput AI pipeline capable of identifying surface anomalies (such as cracks, scratches, and structural deformities) on manufactured products in real time.

Milestone 1 focuses on establishing the core platform infrastructure. This includes designing a full-stack architecture, implementing secure **Role-Based Access Control (RBAC)** via **JSON Web Tokens (JWT)**, integrating standard industrial benchmark datasets, and deploying a live **REST API** ingestion gateway paired with an interactive multi-role frontend dashboard.

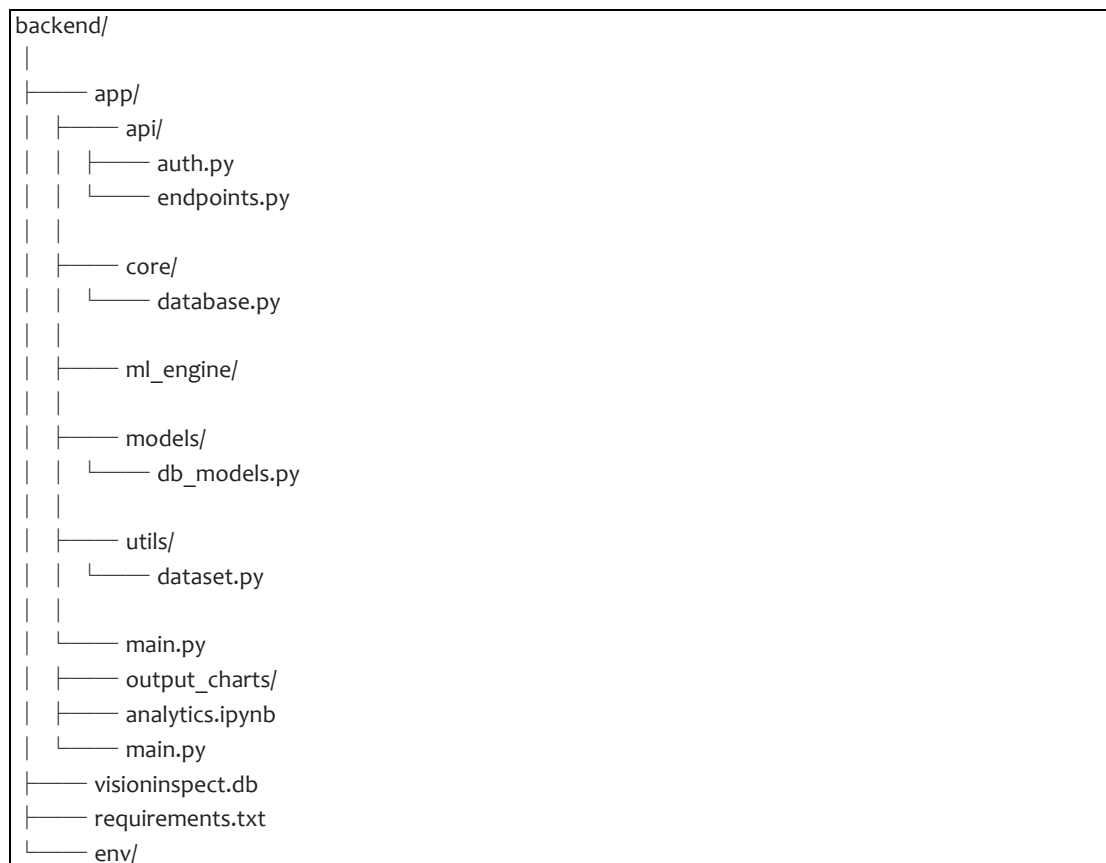


Fig 1: Backend project structure showing API, database , model and utility modules

2 Dataset Used

The project utilizes the global industrial benchmark MVTec Anomaly Detection (MVTec AD) dataset.

Original Scope: The dataset contains thousands of high-resolution industrial product images across 15 distinct manufacturing categories (e.g., bottles, capsules, metal nuts, cables, and transistors).

Data Structure: Each product category is divided into clean, defect-free control samples (good training directory) and specific anomaly classes (such as crack, scratch, broken_large, and contamination in the test directory).

Milestone 1 Data Pipeline (utils/dataset.py): To facilitate automated ingestion, a custom PyTorch Dataset and DataLoader utility was engineered.

The loader systematically traverses folder hierarchies, maps defect names to numerical labels (**0** for **good**, **1** for **anomalous**), and applies uniform image standardization (RGB channel conversion and resizing) to prepare raw data for downstream neural network ingestion.

3 Environment Setup

The platform was developed using a decoupled full-stack architecture, executed locally within an isolated Python virtual environment (env) for the backend and a Node.js runtime for the frontend. The implementation was conducted using Python 3.13 and React.js within Visual Studio Code.

Key Libraries and Technical Configurations:

- **FastAPI & Uvicorn:** Serves as the high-performance asynchronous backend framework and ASGI server, handling concurrent image stream uploads and CORS middleware communication.

- **SQLAlchemy & SQLite:** Manages relational database persistence (visioninspect.db), structurally mapping user authentication credentials and logging comprehensive inspection metadata.
- **PyJWT & Passlib (Bcrypt):** Power the authentication security layer, executing SHA-256 password hashing and token-based role authorization.
- **PyTorch (torch, torchvision):** Initialized within the data utility module to structure image tensors and batch loading for upcoming ML models.
- **OpenCV (opencv-python) & Pillow (PIL):** Essential computer vision libraries utilized for binary stream verification, file format validation, and image dimension extraction.
- **React.js, Vite, & Tailwind CSS:** Drive the frontend interactive interface, utilizing modern utility-first styling and axios for HTTP API telecommunications.

The image shows a split terminal window with two panels. The left panel displays the output of a FastAPI application running with Uvicorn. It shows the application startup process, including directory watching, reloader process starting, and server process starting. The right panel shows the output of a Vite React frontend running in development mode. It displays the Vite version (v8.1.3) and the local development URL (http://localhost:5173/).

```

nothing added to commit but untracked files present (use "git add" to track)
(env) PS C:\Ucd backend
(env) PS C:\Users\KANNA\OneDrive\Desktop\visioninspect-ai\backend> uvicorn app.main
:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\KANNA\\OneDrive\\Desktop\\visioninspect-ai\\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [15732] using StatReload
INFO: Started server process [15940]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:54738 - "OPTIONS /api/auth/login HTTP/1.1" 200 OK
INFO: 127.0.0.1:54738 - "POST /api/auth/login HTTP/1.1" 200 OK
INFO: 127.0.0.1:59838 - "POST /api/auth/login HTTP/1.1" 200 OK
INFO: 127.0.0.1:64197 - "POST /api/upload/ HTTP/1.1" 200 OK
INFO: 127.0.0.1:51474 - "POST /api/auth/login HTTP/1.1" 200 OK
[ ]

PS C:\Users\KANNA\OneDrive\Desktop\visioninspect-ai> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\KANNA\OneDrive\Desktop\visioninspect-ai\backend\env\Scripts\Activate.ps1)
(env) PS C:\Users\KANNA\OneDrive\Desktop\visioninspect-ai> cd frontend
(env) PS C:\Users\KANNA\OneDrive\Desktop\visioninspect-ai\frontend> npm run dev

> frontend@0.0.0 dev
> vite

VITE v8.1.3 ready in 1352 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
[ ]

```

Split terminal execution displaying active FastAPI backend (Uvicorn) and Vite React frontend servers.

4 System Architecture & Role-Based Access Control (RBAC)

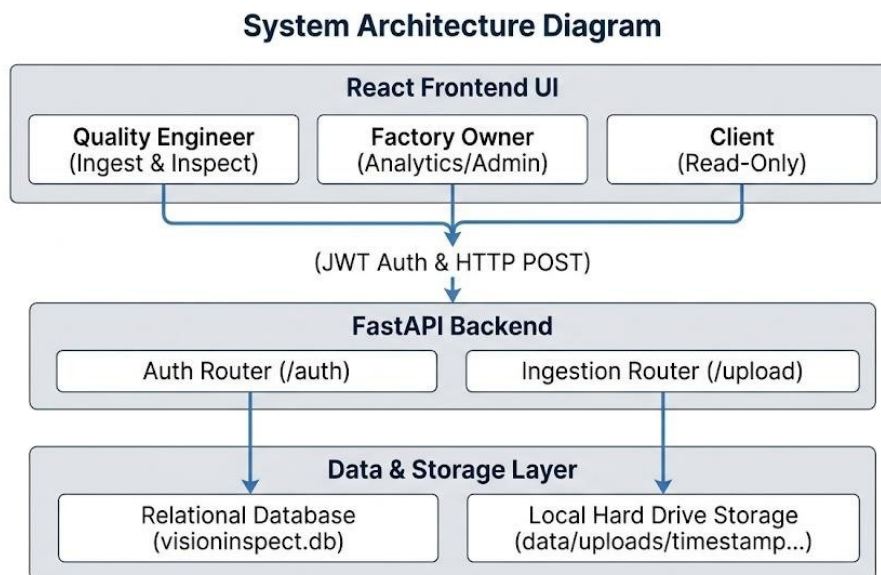
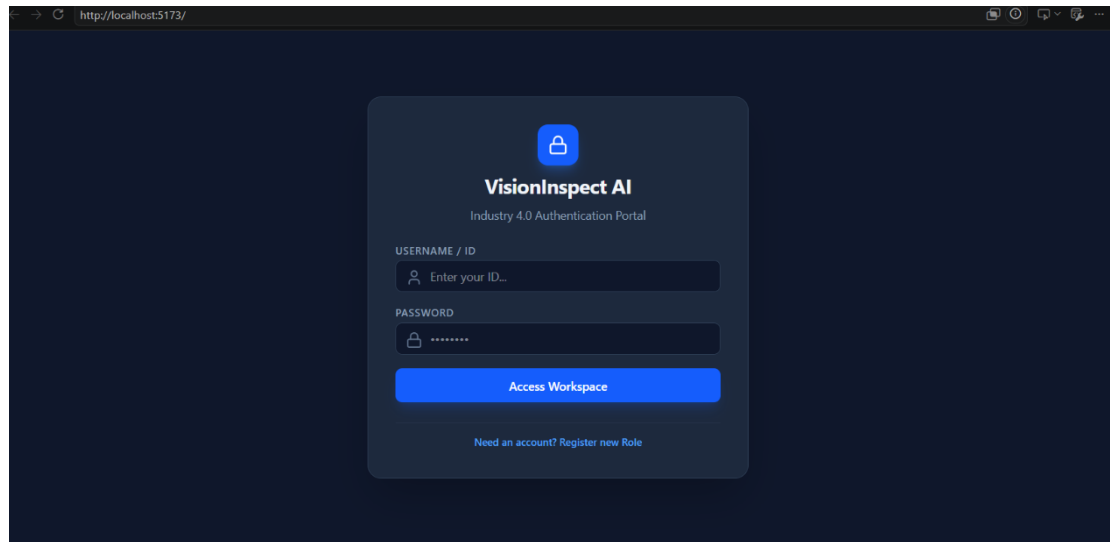


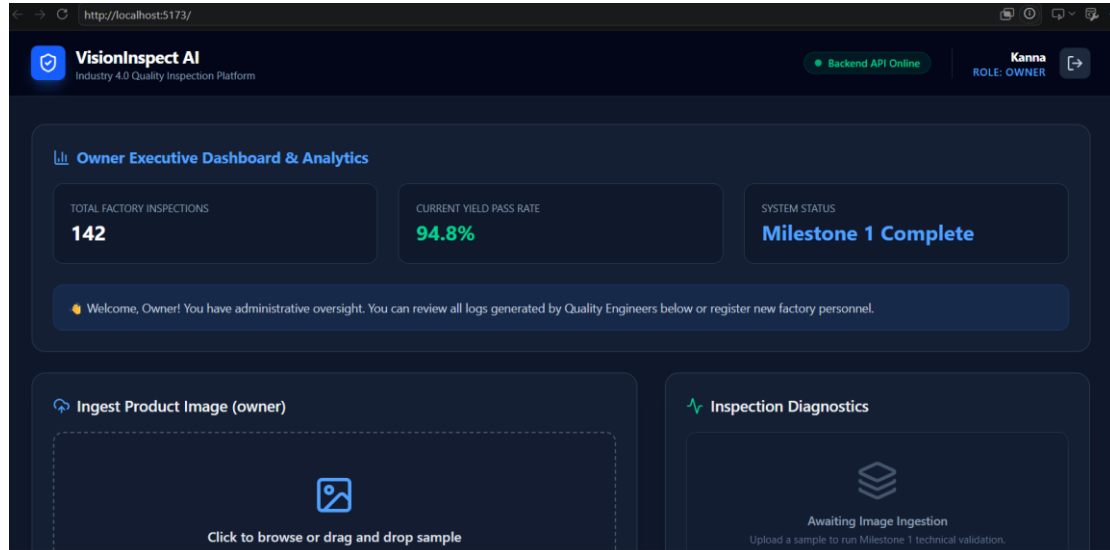
Fig 2 : Backend Architecture and communication flow

1. **Quality Engineer (Active Inspector):** Granted operational permissions to access the image dropzone, ingest product samples, run technical prechecks, and log diagnostic results.
2. **Factory Owner / Supervisor (Executive Admin):** Receives full oversight capabilities, including access to global factory analytics (total inspection volume, yield pass rates), system health telemetry, and historical audit logs.
3. **Client / Buyer (Verification Portal):** Restricted to a read-only interface to independently verify batch integrity checks and review quality assurance reports without alteration permissions.

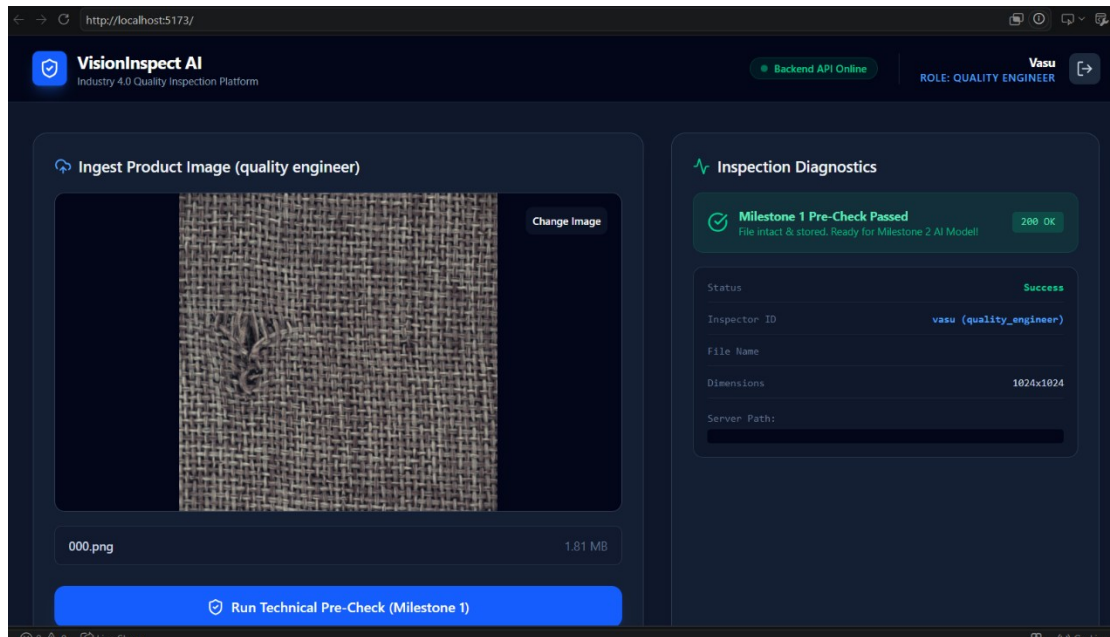
The Above Images Shows About the Dashboard of the Website



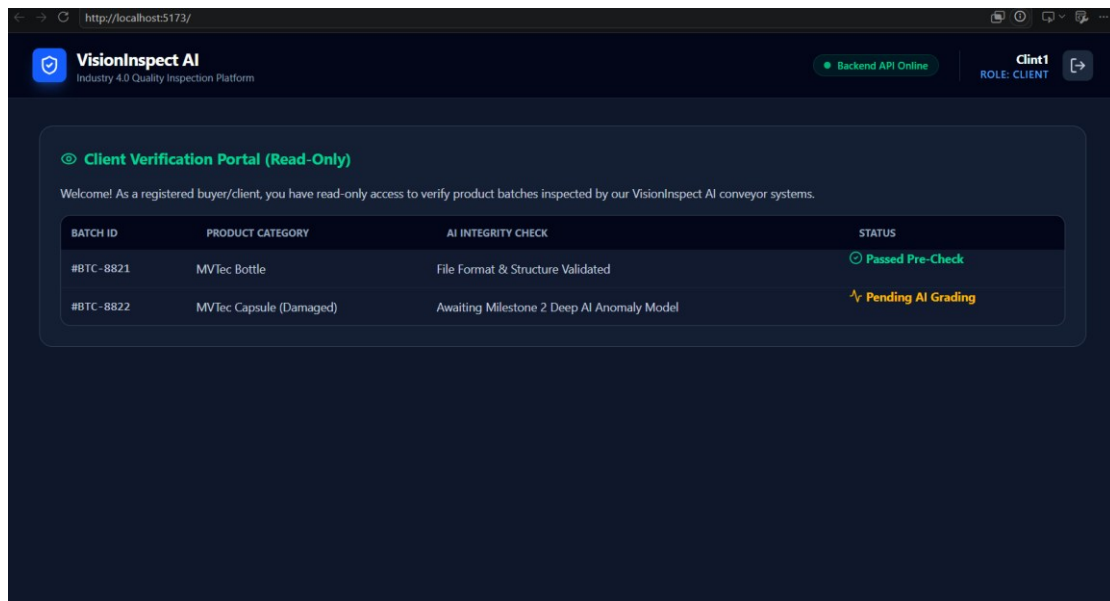
The VisionInspect AI JWT Authentication Portal.



Owner dashboard to check the insight



Quality Engineer Dashboard



Role-Based UI dynamically rendering the Client Verification Portal with read-only batch telemetry.

id	username	hashed_password	role	created_at
1	Kanna	9aca1a52631a2d31c2b92fd1ef171ed2a15cf00025708b0f...	owner	2026-07-07 10:...
2	clint1	6170969ac963de94d4173265cd85a12a4a082d2676bf01...	client	2026-07-07 10:...
3	vasu	1579e75ab1f9eafff4465c8e11e023abd1897c3ad032dc23...	quality_engineer	2026-07-07 10:...
4	client2	8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca1...	client	2026-07-08 13:...

Visioninspect.db of SQLite Database inspection revealing automated schema generation, role assignment, and encrypted password hashing.

VisionInspect AI Backend 0.1.0 OAS 3.1

Authentication

- POST /api/auth/register Register User
- POST /api/auth/login Login User

Inspection

- POST /api/upload/ Upload Image

default

- GET / Home

Schemas

- Body_upload_image_api_upload_post > Expand all object
- HTTPValidationError > Expand all object
- UserLogin > Expand all object
- UserRegister > Expand all object
- ValidationError > Expand all object

Auto-generated Interactive Swagger API documentation validating backend endpoint routing and multi-part data payload acceptance.

5 Core Implementation: Ingestion Pipeline & Multi-Role UI

The Milestones 1 technical workflow operates through a structured sequence to guarantee data integrity before machine learning analysis:

Frontend Image Upload Flow

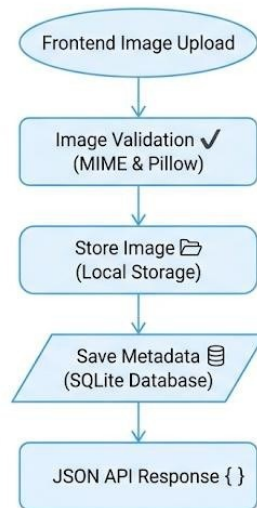


Fig 3 : Image Ingestion and Validation Pipeline

1. **Binary Stream Ingestion:** An image is submitted via the frontend drag-and-drop interface or automated conveyor simulator to the POST `/api/upload/` endpoint.
2. **MIME & Technical Pre-Check:** The server validates that the incoming stream is a legitimate image format (.png, .jpg, .jpeg), blocking malicious file executions.
3. **Pillow Structural Verification:** The image binary is read into memory and processed through `image.verify()` to detect corrupted packets or incomplete transmissions.
4. **Timestamped Disk Archiving:** To prevent naming collisions during high-speed factory batch processing, the file is assigned a unique timestamp (YYYYMMDD_HHMMSS_filename) and permanently committed to the `data/uploads/` directory.
5. **JSON Telemetry Return:** The server generates and returns a structured payload detailing file dimensions, color architecture, storage

paths, and HTTP status codes (200 OK) to update the UI dashboard in real time.

6 Verification & Technical Pre-Check Results

During formal system verification, the platform was evaluated across multiple operating system parameters and network requests:

- ✓ **Authentication Integrity:** Successfully prevented unauthorized route access and correctly dynamically rendered role-specific workspaces upon JWT token validation.
- ✓ **File Ingestion Rigor:** Tested against valid industrial samples (015.png, 001.png) and intentionally damaged or corrupted files. The backend successfully committed valid files to disk while returning 400 Bad Request exceptions for invalid file formats.
- ✓ **Database Generation:** Verified the automated generation and schema population of users and inspections tables within the SQLite engine upon application boot (create_all).

7 Conclusion and Next Steps

Milestone 1 successfully establishes the foundational data, web, and security architecture required for an enterprise-grade Industry 4.0 inspection platform. By bridging React frontend interfaces with an asynchronous FastAPI backend and automated PyTorch data loaders, the platform reliably ingests, validates, and stores industrial image streams.